

Name – Deepanshu

Roll No – 2401010133

Section – C

Assignment no - 2

Subject – ADA LAB

Link- <https://github.com/001-Deepanshu/ADA-LAB-ASSIGNMENT-2>

hello.py > ...

```
1  import time
2  import random
3  import itertools
4  import matplotlib.pyplot as plt
5
6  plt.style.use('ggplot')
7
8  def measure_time(func, *args, runs=3):
9      total = 0
10     for _ in range(runs):
11         start = time.time()
12         func(*args)
13         total += (time.time() - start)
14     return total / runs
15
16  def merge_sort(arr):
17     if len(arr) <= 1:
18         return arr
19     mid = len(arr)//2
20     left = merge_sort(arr[:mid])
21     right = merge_sort(arr[mid:])
22     return merge(left, right)
23
24  def merge(left, right):
25     result = []
26     i = j = 0
27     while i < len(left) and j < len(right):
28         if left[i] < right[j]:
29             result.append(left[i])
30             i += 1
31         else:
32             result.append(right[j])
33             j += 1
34     result.extend(left[i:])
35     result.extend(right[j:])
36     return result
37
38  def quick_sort(arr):
39     if len(arr) <= 1:
40         return arr
41     pivot = arr[len(arr)//2]
42     left = [x for x in arr if x < pivot]
43     mid = [x for x in arr if x == pivot]
44     right = [x for x in arr if x > pivot]
45     return quick_sort(left) + mid + quick_sort(right)
```

hello.py > ...

```
47 sizes = [100, 500, 1000, 2000]
48
49 merge_times = []
50 quick_times = []
51 builtin_times = []
52
53 for n in sizes:
54     arr = random.sample(range(n*10), n)
55
56     merge_times.append(measure_time(merge_sort, arr))
57     quick_times.append(measure_time(quick_sort, arr))
58     builtin_times.append(measure_time(sorted, arr))
59
60
61 plt.figure()
62 plt.plot(sizes, merge_times, label="Merge Sort")
63 plt.plot(sizes, quick_times, label="Quick Sort")
64 plt.plot(sizes, builtin_times, label="Built-in Sort")
65 plt.xlabel("Input Size")
66 plt.ylabel("Time")
67 plt.title("Sorting Algorithm Comparison")
68 plt.legend()
69 plt.show()
70
71 # Fractional Knapsack
72 def fractional_knapsack(weights, values, capacity):
73     ratio = [(values[i]/weights[i], weights[i], values[i]) for i in range(len(weights))]
74     ratio.sort(reverse=True)
75
76     total_value = 0
77     for r, w, v in ratio:
78         if capacity >= w:
79             capacity -= w
80             total_value += v
81         else:
82             total_value += r * capacity
83             break
84     return total_value
85
86 # Activity Selection
87 def activity_selection(start, finish):
88     activities = sorted(zip(start, finish), key=lambda x: x[1])
89     selected = [activities[0]]
90
91     for i in range(1, len(activities)):
```

```

91     for i in range(1, len(activities)):
92         if activities[i][0] >= selected[-1][1]:
93             selected.append(activities[i])
94     return selected
95
96 # Example execution
97 weights = [10, 20, 30]
98 values = [60, 100, 120]
99 capacity = 50
100
101 print("Fractional Knapsack Result:", fractional_knapsack(weights, values, capacity))
102
103 start = [1, 3, 0, 5, 8, 5]
104 finish = [2, 4, 6, 7, 9, 9]
105
106 print("Selected Activities:", activity_selection(start, finish))
107
108 # 0/1 Knapsack
109 def knapsack_dp(weights, values, capacity):
110     n = len(values)
111     dp = [[0]*(capacity+1) for _ in range(n+1)]
112
113     for i in range(1, n+1):
114         for w in range(capacity+1):
115             if weights[i-1] <= w:
116                 dp[i][w] = max(dp[i-1][w], values[i-1] + dp[i-1][w - weights[i-1]])
117             else:
118                 dp[i][w] = dp[i-1][w]
119
120     return dp[n][capacity]
121
122
123 # Fibonacci DP
124 def fib_dp(n):
125     dp = [0, 1]
126     for i in range(2, n+1):
127         dp.append(dp[i-1] + dp[i-2])
128     return dp[n]
129
130
131 print("0/1 Knapsack:", knapsack_dp(weights, values, capacity))
132 print("Fibonacci DP (10):", fib_dp(10))
133
134 def tsp_bruteforce(graph):

```

```
134 def tsp_bruteforce(graph):
135     n = len(graph)
136     vertices = list(range(n))
137     min_path = float('inf')
138
139     for perm in itertools.permutations(vertices[1:]):
140         path = [0] + list(perm) + [0]
141         cost = 0
142         for i in range(len(path)-1):
143             cost += graph[path[i]][path[i+1]]
144         min_path = min(min_path, cost)
145
146     return min_path
147
148 # Generate random graph
149 def generate_graph(n):
150     graph = [[random.randint(10, 100) if i != j else 0 for j in range(n)] for i in range(n)]
151     return graph
152
153
154 sizes = [3, 4, 5, 6]
155 tsp_times = []
156
157 for n in sizes:
158     graph = generate_graph(n)
159     tsp_times.append(measure_time(tsp_bruteforce, graph, runs=1))
160
161
162 plt.figure()
163 plt.plot(sizes, tsp_times, marker='o')
164 plt.xlabel("Number of Cities")
165 plt.ylabel("Time")
166 plt.title("TSP Exponential Growth")
167 plt.show()
```

Fractional Knapsack Result: 240.0

Selected Activities: [(1, 2), (3, 4), (5, 7), (8, 9)]

0/1 Knapsack: 220

Fibonacci DP (10): 55

